

Spring Bean Configuration

1. What is Bean Configuration?

A bean is an object that is created and managed by the Spring IoC container.

Bean configuration is the process of telling Spring

- Which classes should become beans
- How to create them.
- How to connect them with other beans (DI).

Without bean configuration, Spring does not know which objects it should manage.

Example

Suppose you have these classes:

```
Student  
Course  
Teacher
```

Unless you configure them, Spring treats them as normal Java classes.

After configuration, Spring automatically creates and manages them.

Ways to Configure Beans

Spring supports three ways to configure beans:

1. XML Configuration
2. Annotation-Based Configuration (Most Common)
3. Java-Based Configuration

2. Annotation-Based Bean Configuration

Instead of writing XML, we place annotations directly on Java classes.

Spring scans the project, finds these annotations, and automatically creates beans.

Advantages:

- Less XML
- Cleaner code
- Easier to maintain
- Preferred in Spring Boot

Example

```
@Component
public class Student {
}
```

Explanation

`@Component` tells Spring:

| "Create and manage an object of this class."

When the application starts, Spring automatically creates a `Student` bean.

3. Component Scanning

spring needs to know where to look for annotated class.

This process is called **component scanning**.

Example

```
@ComponentScan("com.example")
```

Explanation

Spring scans:

```
com.example
├── Student
├── Employee
├── Teacher
└── Course
```

If these classes contain annotations like `@Component`, Spring creates beans for them automatically.

4. **Stereotype Annotations**

It is specialized versions of `@component`

They tell spring what type of bean that the class represents

@Component

General-purpose bean.

```
@Component
public class Student {
}
```

Used when the class does not belong to a specific layer.

(b) @Service

Represents the **Business Logic Layer**.

```
@Service
public class StudentService {
}
```

Used for:

- Business logic

- Calculations
 - Processing
-

(c) @Repository

Represents the **Data Access Layer (DAO)**.

```
@Repository
public class StudentRepository {
}
```

Used for:

- Database operations
- CRUD operations

Spring also converts database exceptions into Spring exceptions.

(d) @Controller

Represents the **Presentation Layer**.

```
@Controller
public class StudentController {
}
```

Used to handle web requests and return views.

5. Java-Based Configuration

Instead of XML, we create a Java class that tells Spring how to create beans.

This is called **Java-Based Configuration**.

Step 1: Configuration Class

```
@Configuration
public class AppConfig {
}
```

Explanation

`@Configuration` tells Spring:

| "This class contains bean definitions."

Step 2: Create a Bean

```
@Bean
public Student student() {
    return new Student();
}
```

Explanation

`@Bean` tells Spring:

| "The object returned by this method should become a Spring Bean."

Spring executes the method only once (by default) and stores the object in the IoC Container.

Complete Example

```
@Configuration
public class AppConfig {

    @Bean
    public Student student() {
```

```
        return new Student();  
    }  
}
```

Now Spring manages the `Student` object.

6. Mixing XML and Java-Based Configuration

Spring allows XML configuration and Java configuration to work together.

This is useful when:

- Migrating old Spring projects.
- Some beans are defined in XML.
- New beans are defined using Java configuration.

Example

XML:

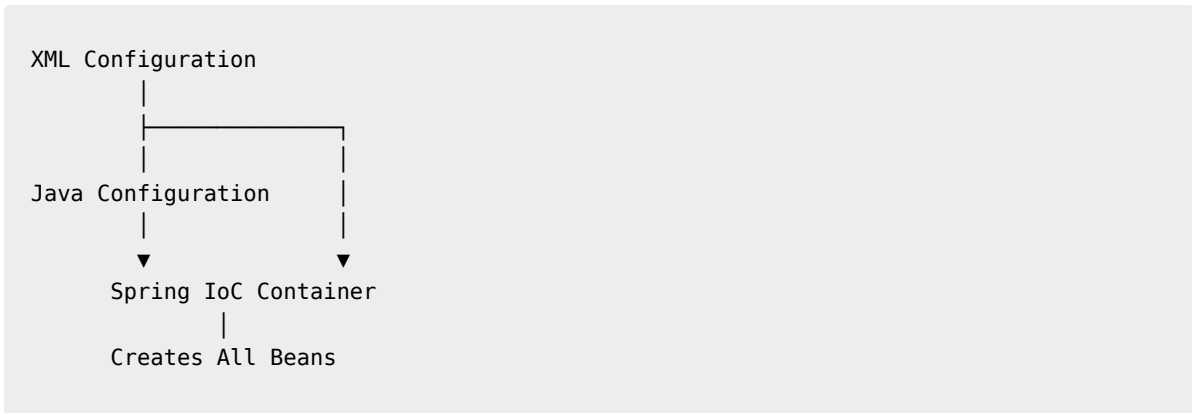
```
<bean id="student"  
      class="com.example.Student"/>
```

Java:

```
@Configuration  
public class AppConfig {  
  
    @Bean  
    public Teacher teacher() {  
        return new Teacher();  
    }  
}
```

Spring loads both configurations and manages all beans together.

XML + Java Configuration Flow



XML vs Annotation vs Java Configuration

XML Configuration	Annotation Configuration	Java Configuration
Beans defined in XML	Beans defined using annotations	Beans defined using Java methods
More XML code	Less code	No XML required
Older approach	Most common in Spring Boot	Used for custom bean creation
Harder to maintain	Easy to maintain	Flexible and type-safe

Interview Definitions

Bean Configuration

Bean Configuration is the process of defining how Spring should create, configure, and manage beans.

@Component

`@Component` is a generic stereotype annotation that tells Spring to automatically create and manage an object as a bean.

Component Scanning

Component Scanning is the process by which Spring automatically searches specified packages for classes annotated with stereotype annotations and registers them as beans.

@Configuration

`@Configuration` indicates that a class contains bean definitions for the Spring IoC Container.

@Bean

`@Bean` marks a method whose returned object should be managed as a Spring Bean.

@Service

Used for the business logic layer.

@Repository

Used for the data access layer and database operations.

@Controller

Used for handling web requests in Spring MVC applications.

Quick Revision

```
Bean Configuration
|
|— XML Configuration
|— Annotation Configuration
|   |— @Component
|   |— @Service
|   |— @Repository
```

```
    |      └─ @Controller
    |
    └─ Java Configuration
          └─ @Configuration
                └─ @Bean

Component Scan
  |
  Finds Annotated Classes
    |
    Creates Spring Beans
```